

Frontend Technical Documentation

1) Overview & Scope

This document describes the **frontend** of a team-based task management system. It covers runtime configuration, architecture, security, routing, data contracts, realtime behavior, build/deploy, testing, quality attributes, and maintenance. The frontend is a **React 18 + TypeScript** Single-Page Application (SPA) built with **Vite**. It communicates with three HTTP services (Auth, Team, Task) via a gateway and maintains a single shared **WebSocket** for realtime task events. ([Aalto Doc](#))

Primary user journeys:

- Register, log in, and persist a session.
 - Create and manage teams and their members.
 - Create, update, complete, and delete tasks (team-scoped and personal views).
 - View a dashboard ("Welcome"), a cross-team task list ("Tasks"), and completed tasks.
 - Administrative management of users and teams.
-

2) Technology Stack

- **Framework:** React 18, TypeScript
 - **Build Tool:** Vite
 - **Routing:** React Router v6
 - **HTTP:** Axios (shared instance) for Auth & Task APIs; `fetch` wrapper for Team API
 - **State:** Local component state + `localStorage` for session
 - **Realtime:** Native WebSocket client + custom hook (`useRealtime`)
 - **Styling:** Inline CSS-in-JS objects (no design system)
-

3) Source Layout (Frontend)

```
src/  
  api/  
    admin.ts      # Admin endpoints against Auth service  
    auth.ts       # Public auth endpoints (login/register) via Axios  
    http.ts       # Shared Axios client, token injectors & refresh
```

```
logic
  task.ts          # Task endpoints via Axios
  team.ts          # Team endpoints via fetch + auth wrapper
auth/
  session.ts       # isAuthenticated(), currentUser(), logout()
components/
  CreateTeamButton.tsx
  LoginForm.tsx
  Navbar.tsx
  RegisterForm.tsx
layouts/
  AppLayout.tsx    # Navbar + WS lifecycle
  AuthLayout.tsx   # Simple container for public routes
pages/
  CompletedPage.tsx
  LoginPage.tsx
  NotFound.tsx
  RegisterPage.tsx
  WelcomePage.tsx
admin/
  TeamsAdminPage.tsx
  TeamAdminDetailPage.tsx
  UsersAdminPage.tsx
tasks/
  DetailedTaskPage.tsx
  TasksPage.tsx
teams/
  CreateTeamPage.tsx
  TeamTasksPage.tsx
  TeamsDetailedPage.tsx
  TeamsPage.tsx
realtime/
  RealtimeRoot.tsx
  eventPatch.ts
  useRealtime.ts
  ws.ts
routes/
  RequireAdmin.tsx
  RequireAuth.tsx
App.tsx
main.tsx
```

Note: Admin routes are guarded separately; by default they are **not** rendered under `AppLayout`, so the navbar is not shown for `/admin/*` unless you nest them under `AppLayout`. ([DIVA Portal](#))

4) Runtime Configuration

4.1 Environment Variables

- `VITE_AUTH_API_BASE_URL` (default: `"/api"`): Axios base URL for `http` client (`api/http.ts`).
- `VITE_WS_URL` (default: `"/ws"`): WebSocket base path (relative; resolved by gateway/proxy). ([IETF Datatracker](#))

4.2 Expected Gateway Paths (Prod & Dev)

- `"/api"` → HTTP gateway to Auth, Task, Team services (rewritten internally).
- `"/ws"` → WebSocket gateway for task events.

The frontend constructs **relative** URLs so the same build runs in dev and prod behind a single reverse proxy. ([etd.auburn.edu](#))

5) Architecture

5.1 Routing & Layouts

- **Public** (`AuthLayout`): `/` (Login), `/register`.
- **Private** (`AppLayout` + `RequireAuth`):
 - `/welcome` (dashboard)
 - `/tasks` (cross-team list), `/tasks/:id` (detail)
 - `/teams` (my teams), `/teams/new`, `/teams/:id` (team board),
`/teams/:id/manage` (team admin for owners/admins)
 - `/completed` (completed tasks)
 - `*` (NotFound)
- **Admin** (`RequireAdmin`): `/admin/users`, `/admin/teams`, `/admin/teams/:id`.

`AppLayout` renders a persistent **Navbar** and opens a single **WebSocket** connection for private routes. (trepo.tuni.fi)

5.2 Session & Guards

- **Storage keys:**
 - `accessToken` (JWT), `refreshToken` (JWT), `currentUser` (minimal user object).
- **Guards:**
 - `RequireAuth`: redirects to `/` with `state.from` if not authenticated.
 - `RequireAdmin`: accepts either `u.isAdmin === true` or `u.role?.toLowerCase() === "admin"`; otherwise redirects to `/welcome`. ([DIVA Portal](#))

5.3 Data Access

- **Axios (`api/http.ts`)**: Shared client with:
 - Request interceptor: inject `Authorization: Bearer <accessToken>`.
 - Response interceptor: on `401`, single-flight `refresh( /auth/refresh )`, retry original request; on failure → `logout()`. ([arXiv](#))
- **fetch wrapper (`api/team.ts`)**: `authFetch` sets JWT header and JSON headers; throws enriched Errors on non-2xx. (etd.auburn.edu)

5.4 Realtime (WebSocket)

- **Singleton client (`realtime/ws.ts`)**:
 - URL: `${VITE_WS_URL} ?? "/ws"?userId=<id>&token=<accessToken>`
 - Heartbeat ping every 25s.
 - Exponential backoff reconnect (cap 30s).
 - Normalizes backend messages to `TaskEvent` regardless of shape.
- **Hook (`realtime/useRealtime.ts`)**:
 - Subscribes to events with optional throttle (default 150ms).
 - Cleans up timers and unsubscribes on unmount.
- **Normalization (`realtime/eventPatch.ts`)**:

- Converts event payloads (flat or `{ task: {...} }`) to `Partial<Task>` with strict field/type checks. ([IETF Datatracker](#))
-

6) Feature Coverage

6.1 Authentication

- **Login:** `POST /auth/login` → stores tokens; if user missing, `GET /auth/me`; persist `currentUser`; redirect `/welcome`.
- **Register:** `POST /auth/register`; prompts to log in afterwards.
- **Session:** token refresh on 401; `logout()` clears storage and navigates to `/`. ([arXiv](#))

6.2 Tasks

- **Cross-team list** (`/tasks`): `GET /tasks` for current user, realtime updates, toggle complete, delete.
- **Team board** (`/teams/:id`): list team tasks, quick add, toggle, delete; realtime subscription filtered by `teamId`.
- **Task detail** (`/tasks/:id`): load, edit (title/description/priority/due), assign member, toggle completion, delete; merges incoming realtime patches while respecting local edits. ([DIVA Portal](#))

6.3 Teams

- **My teams** (`/teams`): list teams for current user.
- **Create team** (`/teams/new`): creates team; best-effort set creator as `OWNER`; redirects to team board.
- **Team manage (owner/admin)** (`/teams/:id/manage`): edit team meta, invite by numeric `userId`, remove member, delete team. ([trepo.tuni.fi](#))

6.4 Administration

- **Users** (`/admin/users`): list/create/update/delete; supports both `role` and `isAdmin`.
- **Teams** (`/admin/teams`): list all teams.
- **Team detail** (`/admin/teams/:id`): edit team, list members, add member (role), change member role, remove member. ([trepo.tuni.fi](#))

6.5 Completed & Welcome

- **Completed** (`/completed`): derives completed tasks from `listMyTasks()` .
 - **Welcome** (`/welcome`): dashboard with “My Tasks” and “My Teams” fetched in parallel; robust shape checks. ([DIVA Portal](#))
-

7) API Contracts (Frontend Expectations)

All paths are relative to the gateway prefix `"/api"` .

7.1 Auth Service

- `POST /auth/login → { accessToken, refreshToken, user? }`
- `POST /auth/register → UserResponse`
- `POST /auth/refresh → { accessToken, refreshToken? }`
- `GET /auth/me → UserResponse`
- **Admin:**
 - `GET /auth/admin/users → AdminUser[]`
 - `POST /auth/admin/users → AdminUser`
 - `PATCH /auth/admin/users/:id → AdminUser`
 - `DELETE /auth/admin/users/:id → 204`

7.2 Task Service

- `GET /tasks → Task[] | { tasks|data|items: Task[] }`
- `GET /tasks/teams/:teamId/tasks → list tasks for a team`
- `POST /tasks/teams/:teamId/tasks → create task (title required)`
- `GET /tasks/:id → Task`
- `PUT /tasks/:id → update`
- `DELETE /tasks/:id → 204`
- `PUT /tasks/:id/assignee → { assigneeId } → Task`
- `POST /tasks/:id/complete → { completed } → Task`

7.3 Team Service

- `POST /teams → Team`

- GET /users/:userId/teams → Team[]
- GET /teams/:teamId → Team
- GET /teams/:teamId/members → TeamMember[]
- PUT /teams/:teamId → Team
- DELETE /teams/:teamId → 204
- POST /teams/:teamId/members → { userId, role } → TeamMember
- DELETE /teams/:teamId/members/:userId → 204
- PATCH /teams/:teamId/members/:userId → { role } → TeamMember
- (Optional) POST /teams/:teamId/members/invite → { username, role } → TeamMember (if backend supports invites by username)

7.4 Data Models (selected)

```
type Task = {
  id: number;
  teamId: number;
  title: string;
  description?: string;
  priority?: "low" | "medium" | "high";
  due?: string; // YYYY-MM-DD
  assigneeId?: number | null;
  completed: boolean;
  createdAt?: string;
  updatedAt?: string;
};
```

```
type Team = { id: number; name: string; description?: string };
type TeamRole = "OWNER" | "ADMIN" | "MEMBER";
type TeamMember = { id: number; username: string; role: TeamRole };
```

```
type UserResponse = {
  id: number; username: string; email: string;
  firstName?: string; lastName?: string;
  role: string; isActive: boolean;
  createdAt: string; updatedAt: string;
};
```

```
type AdminUser = {
  id: number; username: string; email?: string;
  role?: string; isAdmin?: boolean;
```

```
    createdAt?: string; updatedAt?: string;
  };
```

8) Realtime Protocol

8.1 Event Envelope (normalized)

```
type TaskEventType = "task.created" | "task.updated" | "task.deleted" | "

interface TaskEvent {
  eventType: TaskEventType;
  taskId: number;
  teamId: number;
  actorId: number;
  creatorId: number;
  assigneeId?: number | null;
  timestamp: string; // ISO
  payload?: Record<string, unknown>;
}
```

The client adapts multiple backend formats (e.g., { `type`, `data` } vs { `eventType`, `payload` }) and canonicalizes event type strings (snake/camel → dotted lowercase). Heartbeats are JSON { `type: "ping"` } .([IETF Datatracker](#))

8.2 Payload Normalization

`eventPatch.ts` produces a **strict** `Partial<Task>` by validating keys and types. It accepts flat payloads or { `task: { ... }` }. Fields outside the `Task` schema are ignored. ([gupea.ub.gu.se](#))

8.3 Subscription Semantics

- `useRealtime(handler, { throttleMs=150 })` throttles bursts to reduce re-renders.
 - Pages filter by `teamId` or `taskId` as needed.
 - When a task is deleted and the user is on its detail page, the UI navigates away. ([DIVA Portal](#))
-

9) Security

- **JWT storage:** `accessToken` , `refreshToken` in `localStorage` .
- **HTTP auth:** `Authorization: Bearer <accessToken>` via Axios interceptor or `authFetch` .
- **Refresh strategy:** Single-flight refresh with retry queue; on failure, `logout()` .
- **WS auth:** `userId` and `token` included as query parameters (requires TLS and server validation).
- **Guardrails:** No admin UI without `RequireAdmin` ; no private routes without `RequireAuth` . ([IETF Datatracker](#))

For stronger WS secrecy, consider header-based auth or one-time WS tokens (backend change). Also ensure secure WebSocket usage (`wss://`) and origin checks to mitigate Cross-Site WebSocket Hijacking. ([IETF Datatracker](#))

10) Error Handling

- **HTTP:** Non-2xx responses become `Error` with status and body text (fetch) or Axios error; surfaced inline to the user.
 - **Shape validation:** Defensive checks in `task.ts` unwrap list responses and detect proxy HTML fallbacks.
 - **UI:** Mutations render disabling states and confirm destructive actions. Most mutations optimistically update local lists after success. (etd.auburn.edu)
-

11) UI/UX & Accessibility

- **Consistency:** Shared button/input style objects ensure a minimal coherent look.
 - **Feedback:** Loading spinners (`Loading...`), inline error messages, success-path navigation.
 - **Keyboard/ARIA:** Basic labels present; further ARIA roles and focus styles recommended for accessibility parity.
 - **Responsiveness:** Works on desktop; limited mobile optimizations (no dedicated breakpoints yet). ([DIVA Portal](#))
-

12) Performance & Reliability

- **WebSocket:** Heartbeats, reconnect backoff (up to 30s), singleton connection to avoid duplicates.
- **Rendering:** Throttled event delivery avoids re-render storms.
- **Network:** Parallel fetches for dashboard; minimal over-fetching. (gupea.ub.gu.se)

Planned improvements:

- Route-level code splitting with `React.lazy`.
 - Debounce for text inputs where applicable.
 - Memoization for long lists and derived computations. (trepo.tuni.fi)
-

13) Build, Run & Deployment

13.1 Development

```
npm install
npm run dev
```

Requirements:

- Gateway/proxy exposes `"/api"` and `"/ws"` to local services. ([IETF Datatracker](https://datatracker.ietf.org/))

13.2 Production

```
npm run build
# serve dist/ behind a reverse proxy that forwards:
#   /api → HTTP services (auth/task/team)
#   /ws  → WebSocket service
```

- Ensure SPA fallback to `index.html` for client routes.
 - Keep environment variables consistent with gateway paths. (etd.auburn.edu)
-

14) Testing & QA

14.1 Manual Smoke Tests

1. **Auth:** Register → Login → Reload → Session persists; then Logout.
2. **Teams:** Create team → Appears in “My Teams”; open team board.
3. **Tasks:** Add task in team → Toggle complete → Delete; verify realtime reflection in a second browser window.
4. **Task Detail:** Edit fields, assign member, toggle completion, delete.
5. **Admin Users:** Create, update (role/isAdmin), delete; relogin as affected user if needed.

- 6. **Admin Teams:** Edit settings; add/remove members; change roles.

14.2 Automation Targets (next)

- Unit: `api/http.ts` (refresh queue), `realtime/eventPatch.ts` (payload normalization).
 - Component: `TasksPage`, `TeamTasksPage` (realtime reconciliation).
 - E2E: Login, create team, add task, complete task, admin CRUD. (trepo.tuni.fi)
-

15) Logging & Diagnostics

- **WS debug:** `window.__DEBUG_WS__` logs incoming frames; `AppLayout` logs status transitions in dev.
 - **Auth refresh:** Emits a `window` event `auth:token-refreshed` after a successful refresh (for listeners like WS if needed). (gupea.ub.gu.se)
-

16) Known Limitations & Risks

- **WS token in query:** Visible in network tooling; rely on TLS and backend validation; consider alternative auth mechanism.
- **Mobile UX:** Limited responsiveness; forms/tables may overflow on small screens.
- **Admin routing:** Admin pages live outside `AppLayout`; navbar and WS are not mounted there unless explicitly nested.
- **No global toast/notification system:** Errors appear inline or `alert()` in some admin pages.
- **No automated tests:** See §14.2 for priorities. ([IETF Datatracker](https://ietf-datatracker.org))

Mitigations:

- Introduce a simple toast system; add breakpoints and semantic focus states; adopt route splitting; add foundational tests. (trepo.tuni.fi)
-

17) Maintenance Checklist

- **Security:** Rotate tokens policy; verify refresh endpoint; consider WS auth improvements.
 - **UX:** Add toasts, responsive layout, accessible focus outlines.
 - **Observability:** WS status indicator in UI (non-dev).
 - **Docs:** Keep §7 API Contracts in sync with backend changes.
 - **Cleanup:** Periodically remove unused components and dead code paths. ([DIVA Portal](https://diva.portal.fi))
-

18) Glossary

- **SPA**: Single-Page Application.
 - **DTO**: Data Transfer Object (TypeScript interface for API payloads).
 - **JWT**: JSON Web Token used for authentication.
 - **Heartbeat**: Periodic message to keep WS connection alive.
 - **Throttle**: Limit event handling rate to reduce renders. ([Aalto Doc](#))
-

19) Route Map (Authoritative)

Public:

/	→ LoginPage
/register	→ RegisterPage

Private (RequireAuth, AppLayout, RealtimeShell):

/welcome	→ WelcomePage
/tasks	→ TasksPage
/tasks/:id	→ DetailedTaskPage
/teams	→ TeamsPage
/teams/new	→ CreateTeamPage
/teams/:id	→ TeamTasksPage
/teams/:id/manage	→ TeamsDetailedPage
/completed	→ CompletedPage
*	→ NotFoundPage

Admin (RequireAdmin):

/admin/users	→ UsersAdminPage
/admin/teams	→ TeamsAdminPage
/admin/teams/:id	→ TeamAdminDetailPage

20) File Index (Key Responsibilities)

- **api/http.ts** : Axios base URL resolution; JWT injection; 401 refresh queue; retry logic.
- **api/auth.ts** : Login, register, error message extraction.
- **api/task.ts** : Task contracts; robust list unwrapping; CRUD; setAssignee; setCompleted.
- **api/team.ts** : Team contracts via **fetch** ; create/list/update/delete; member ops; admin ops; optional invite.

- `api/admin.ts` : Admin user CRUD against Auth service.
- `realtime/ws.ts` : Singleton WS; event adaptation; backoff; heartbeat; subscriber registries.
- `realtime/useRealtime.ts` : React subscription hook with throttle and cleanup.
- `realtime/eventPatch.ts` : Strict extraction of `Partial<Task>` from payloads.
- `routes/RequireAuth.tsx` : Private route guard.
- `routes/RequireAdmin.tsx` : Admin route guard; accepts `role` or `isAdmin`.
- `layouts/AppLayout.tsx` : Navbar; WS connect/cleanup; content container.
- `layouts/AuthLayout.tsx` : Simple public container.
- `components/Navbar.tsx` : Navigation; logout; contextual “Create team” button.
- `pages/...` : See §6 for per-page behaviors. ([DIVA Portal](#))